

Лабораторная работа №8

Двоичный поиск. Хеш-таблицы

Цель работы

1. Реализовать алгоритм двоичного (бинарного) поиска в отсортированном массиве
2. Освоить итеративную и рекурсивную реализации двоичного поиска
3. Изучить понятие хеш-таблицы, хеш-функции и методы разрешения коллизий
4. Реализовать хеш-таблицу с методом цепочек и открытой адресацией
5. Сравнить производительность двоичного поиска и хеш-таблицы

Теоретическое обоснование

1. Двоичный (бинарный) поиск

Двоичный поиск - это алгоритм поиска элемента в отсортированном массиве, который на каждом шаге делит интервал поиска пополам.

Принцип работы:

Находим средний элемент интервала поиска

Сравниваем с искомым значением

Если равно - возвращаем индекс

Если больше - продолжаем поиск в левой половине

Если меньше - продолжаем поиск в правой половине

Визуализация поиска числа 7 в массиве [1, 3, 5, 7, 9, 11, 13]:

[1, 3, 5, 7, 9, 11, 13] → средний = 7? Да! Нашли.

[1, 3, 5, 7, 9] → средний = 5? Меньше, ищем справа

[5, 7, 9] → средний = 7? Нашли.

Сложность:

Характеристика	Значение
Лучший случай	$O(1)$
Средний случай	$O(\log n)$
Худший случай	$O(\log n)$
Память (итеративно)	$O(1)$

Характеристика	Значение
Память (рекурсивно)	$O(\log n)$

Итеративная реализация

```
int binarySearch(const int arr[], int n, int target) {
    int left = 0, right = n - 1;
    while (left <= right) {
        int mid = left + (right - left) / 2; // (left + right) / 2
        if (arr[mid] == target) return mid;
        if (arr[mid] < target) left = mid + 1;
        else right = mid - 1;
    }
    return -1;
}
```

Рекурсивная реализация

```
int binarySearchRecursive(const int arr[], int left, int right, int target) {
    if (left > right) return -1;
    int mid = left + (right - left) / 2;
    if (arr[mid] == target) return mid;
    if (arr[mid] < target)
        return binarySearchRecursive(arr, mid + 1, right, target);
    return binarySearchRecursive(arr, left, mid - 1, target);
}
```

2. Хеш-таблицы

Хеш-таблица - это структура данных, реализующая ассоциативный массив (словарь) и поддерживающая операции вставки, удаления и поиска в среднем за $O(1)$.

Основные компоненты:

Хеш-функция - преобразует ключ в индекс массива

Бакет (bucket) - элемент массива, хранящий данные

Коллизия - ситуация, когда разные ключи дают одинаковый индекс

Сложность:

Операция	Средний случай	Худший случай
Вставка	$O(1)$	$O(n)$
Поиск	$O(1)$	$O(n)$
Удаление	$O(1)$	$O(n)$

3. Хеш-функции

Требования к хеш-функции:

Детерминированность (один ключ $\rightarrow$ один хеш)

Равномерное распределение

Быстрое вычисление

Примеры хеш-функций для целых чисел:

// Простейшая (не рекомендуется)

```
int hash1(int key, int size) {
    return key % size;
}
```

// Для строк (простая)

```
int hashString(const string& key, int size) {
    int hash = 0;
    for (char c : key) hash = (hash * 31 + c) % size;
    return hash;
}
```

4. Метод цепочек (Chaining)

При коллизии элементы с одинаковым хешем хранятся в связном списке.

// Структура узла

```
template <typename K, typename V>
struct Node {
    K key;
    V value;
    Node* next;
    Node(const K& k, const V& v) : key(k), value(v), next(nullptr) {}
};
```

// Хеш-таблица с цепочками

```

template <typename K, typename V>
class HashTableChaining {
private:
    vector<Node<K, V>*> table;
    int size;

    int hash(const K& key) const {
        return key % size;
    }

public:
    HashTableChaining(int s) : size(s), table(s, nullptr) {}

    void insert(const K& key, const V& value) {
        int index = hash(key);
        Node<K, V>* newNode = new Node<K, V>(key, value);
        newNode->next = table[index];
        table[index] = newNode;
    }

    V* find(const K& key) const {
        int index = hash(key);
        Node<K, V>* current = table[index];
        while (current != nullptr) {
            if (current->key == key) return &current->value;
            current = current->next;
        }
        return nullptr;
    }
};

```

5. Открытая адресация (Open Addressing)

При коллизии ищется следующий свободный бакет.

Методы разрешения коллизий:

Метод	Формула	Описание
Линейное пробирование	$(\text{hash} + i) \% \text{size}$	Поиск следующей свободной ячейки
Квадратичное пробирование	$(\text{hash} + i^2) \% \text{size}$	Поиск с квадратичным шагом

Метод	Формула	Описание
Двойное хеширование	$(\text{hash1} + i * \text{hash2}) \% \text{size}$	Вторая хеш-функция

```

template <typename K, typename V>
class HashTableOpenAddressing {
private:
    struct Entry {
        K key;
        V value;
        bool occupied;
        Entry() : occupied(false) {}
    };

    vector<Entry> table;
    int size;

    int hash(const K& key) const {
        return key % size;
    }

public:
    HashTableOpenAddressing(int s) : size(s), table(s) {}

    void insert(const K& key, const V& value) {
        int index = hash(key);
        int i = 0;
        while (table[(index + i) % size].occupied) {
            if (table[(index + i) % size].key == key) break;
            i++;
        }
        int pos = (index + i) % size;
        table[pos].key = key;
        table[pos].value = value;
        table[pos].occupied = true;
    }

    V* find(const K& key) {
        int index = hash(key);
        int i = 0;

```

```

while (table[(index + i) % size].occupied) {
    int pos = (index + i) % size;
    if (table[pos].key == key) return &table[pos].value;
    i++;
}
return nullptr;
}
};

```

6. Сравнение методов разрешения коллизий

Характеристика	Метод цепочек	Линейное пробирование	Квадратичное пробирование
Использование памяти	Дополнительная на указатели	Только массив	Только массив
Кластеризация	Нет	Да (первичная)	Меньше (вторичная)
Производительность	Хорошая	Средняя	Хорошая
Простота реализации	Средняя	Простая	Средняя

Примеры выполнения практической части

Пример 1. Двоичный поиск (базовый уровень)

Задача: Реализовать итеративный и рекурсивный двоичный поиск. Сравнить их производительность.

```

#include <iostream>
#include <chrono>
#include <algorithm>

```

```

using namespace std;
using namespace chrono;

```

```

// Итеративная версия
int binarySearchIter(const int arr[], int n, int target) {

```

```

int left = 0, right = n - 1;
while (left <= right) {
    int mid = left + (right - left) / 2;
    if (arr[mid] == target) return mid;
    if (arr[mid] < target) left = mid + 1;
    else right = mid - 1;
}
return -1;
}

// Рекурсивная версия
int binarySearchRecur(const int arr[], int left, int right, int target) {
    if (left > right) return -1;
    int mid = left + (right - left) / 2;
    if (arr[mid] == target) return mid;
    if (arr[mid] < target)
        return binarySearchRecur(arr, mid + 1, right, target);
    return binarySearchRecur(arr, left, mid - 1, target);
}

// Линейный поиск для сравнения
int linearSearch(const int arr[], int n, int target) {
    for (int i = 0; i < n; i++) {
        if (arr[i] == target) return i;
    }
    return -1;
}

int main() {
    const int N = 1000000;
    int* arr = new int[N];
    for (int i = 0; i < N; i++) arr[i] = i;

    int target = N - 1;

    auto start = high_resolution_clock::now();
    int linearResult = linearSearch(arr, N, target);
    auto end = high_resolution_clock::now();
    long long linearTime = duration_cast<microseconds>(end - start).count();

    start = high_resolution_clock::now();
    int binaryResult = binarySearchIter(arr, N, target);

```

```

end = high_resolution_clock::now();
long long binaryTime = duration_cast<microseconds>(end - start).count();

cout << "Линейный поиск:  " << linearTime << " мкс" << endl;
cout << "Бинарный поиск:  " << binaryTime << " мкс" << endl;

delete[] arr;
return 0;
}

```

Ожидаемый вывод:

Линейный поиск: 1784 мкс
Бинарный поиск: 2 мкс

Пример 2. Хеш-таблица с методом цепочек (средний уровень)

Задача: Реализовать хеш-таблицу для хранения пар (ключ, значение) с методом цепочек.

```

#include <iostream>
#include <vector>
#include <string>

using namespace std;

template <typename K, typename V>
class HashTable {
private:
    struct Node {
        K key;
        V value;
        Node* next;
        Node(const K& k, const V& v) : key(k), value(v), next(nullptr) {}
    };

    vector<Node*> table;
    int size;
    int count;

    int hash(const K& key) const {
        return key % size;
    }
}

```


public:

```
HashTable(int s = 10) : size(s), count(0) {  
    table.resize(size, nullptr);  
}
```

```
~HashTable() {  
    for (int i = 0; i < size; i++) {  
        Node* current = table[i];  
        while (current != nullptr) {  
            Node* temp = current;  
            current = current->next;  
            delete temp;  
        }  
    }  
}
```

```
void insert(const K& key, const V& value) {  
    int index = hash(key);  
    Node* current = table[index];  
    while (current != nullptr) {  
        if (current->key == key) {  
            current->value = value;  
            return;  
        }  
        current = current->next;  
    }  
    Node* newNode = new Node(key, value);  
    newNode->next = table[index];  
    table[index] = newNode;  
    count++;  
}
```

```
bool find(const K& key, V& result) const {  
    int index = hash(key);  
    Node* current = table[index];  
    while (current != nullptr) {  
        if (current->key == key) {  
            result = current->value;  
            return true;  
        }  
        current = current->next;  
    }  
}
```

```

        return false;
    }

    bool remove(const K& key) {
        int index = hash(key);
        Node* current = table[index];
        Node* prev = nullptr;
        while (current != nullptr) {
            if (current->key == key) {
                if (prev == nullptr) table[index] = current->next;
                else prev->next = current->next;
                delete current;
                count--;
                return true;
            }
            prev = current;
            current = current->next;
        }
        return false;
    }

    int getSize() const { return count; }

    void print() const {
        for (int i = 0; i < size; i++) {
            cout << i << ": ";
            Node* current = table[i];
            while (current != nullptr) {
                cout << "(" << current->key << ", " << current->value << ") ";
                current = current->next;
            }
            cout << endl;
        }
    }
};

int main() {
    HashTable<int, string> ht(5);

    ht.insert(10, "десять");
    ht.insert(20, "двадцать");
    ht.insert(30, "тридцать");

```

```

    ht.insert(15, "пятнадцать");
    ht.insert(25, "двадцать пять");

    ht.print();

    string value;
    if (ht.find(20, value)) {
        cout << "Ключ 20: " << value << endl;
    }

    ht.remove(20);
    cout << "После удаления 20:" << endl;
    ht.print();

    return 0;
}

```

Пример 3. Сравнение поиска в массиве и хеш-таблице (продвинутый уровень)

Задача: Сравнить производительность бинарного поиска в отсортированном массиве и поиска в хеш-таблице.

```

#include <iostream>
#include <chrono>
#include <unordered_map>
#include <vector>
#include <algorithm>

using namespace std;
using namespace chrono;

int main() {
    const int N = 1000000;
    vector<int> arr(N);
    unordered_map<int, int> hashMap;

    for (int i = 0; i < N; i++) {
        arr[i] = i;
        hashMap[i] = i;
    }

    vector<int> queries;

```

```

for (int i = 0; i < 10000; i++) queries.push_back(rand() % N);

// Бинарный поиск
auto start = high_resolution_clock::now();
for (int q : queries) {
    binary_search(arr.begin(), arr.end(), q);
}
auto end = high_resolution_clock::now();
long long binaryTime = duration_cast<microseconds>(end - start).count();

// Хеш-таблица
start = high_resolution_clock::now();
for (int q : queries) {
    hashMap.find(q);
}
end = high_resolution_clock::now();
long long hashTime = duration_cast<microseconds>(end - start).count();

cout << "10000 запросов:" << endl;
cout << "Бинарный поиск (O(log n)): " << binaryTime << " мкс" <<
endl;
cout << "Хеш-таблица (O(1) среднее): " << hashTime << " мкс" << endl;

return 0;
}

```

Индивидуальные задания

Часть 1. Двоичный поиск

Каждый вариант содержит задание реализовать двоичный поиск с указанными особенностями:

№ вар.	Тип данных	Особенности
1	int	Итеративная реализация
2	int	Рекурсивная реализация
3	int	Поиск первого вхождения
4	int	Поиск последнего вхождения

№ вар.	Тип данных	Особенности
5	int	Поиск границ (левая и правая)
6	double	Сравнение с погрешностью
7	string	Лексикографический поиск
8	struct Person (по возрасту)	Поиск по ключу
9	int	Поиск в циклически сдвинутом массиве
10	int	Поиск элемента, ближайшего к заданному
11	int	Поиск в массиве, отсортированном по убыванию
12	int	Подсчёт количества вхождений
13	int	Поиск с помощью std::lower_bound
14	int	Поиск с помощью std::upper_bound
15	int	Поиск в массиве с дубликатами (любое вхождение)

Часть 2. Хеш-таблицы

Каждый вариант содержит задание реализовать хеш-таблицу с указанными особенностями:

№ вар.	Метод разрешения коллизий	Тип ключа	Тип значения
1	Метод цепочек	int	int

№ вар.	Метод разрешения коллизий	Тип ключа	Тип значения
2	Линейное пробирование	int	int
3	Квадратичное пробирование	int	int
4	Двойное хеширование	int	int
5	Метод цепочек	string	int
6	Линейное пробирование	string	int
7	Метод цепочек	int	string
8	Линейное пробирование	int	string
9	Метод цепочек	string	string
10	Линейное пробирование	string	string
11	Метод цепочек	int	struct Student
12	Линейное пробирование	int	struct Student
13	Метод цепочек (динамическое расширение)	int	int
14	Линейное пробирование (динамическое расширение)	int	int
15	Метод цепочек + рехеширование	int	int

Часть 3. Сравнительный анализ

Задание для всех вариантов:

1. Реализовать класс хеш-таблицы согласно варианту
2. Реализовать двоичный поиск (итеративно)

3. Сравнить производительность операций поиска для $N = 1000$, 10000 , 100000 элементов:
4. Двоичный поиск в отсортированном массиве
5. Поиск в хеш-таблице
6. Построить таблицу/график зависимости времени от N

Контрольные вопросы

1. Какое условие необходимо для применения двоичного поиска?
2. Какова сложность двоичного поиска? Почему она логарифмическая?
3. В чём разница между итеративной и рекурсивной реализацией двоичного поиска?
4. Чем опасен вариант $mid = (left + right) / 2$? Как правильно вычислять середину?
5. Что такое хеш-таблица? Из каких компонентов она состоит?
6. Какие требования предъявляются к хеш-функции?
7. Что такое коллизия в хеш-таблице? Как с ней бороться?
8. В чём отличие метода цепочек от открытой адресации?
9. Что такое первичная и вторичная кластеризация?
10. Какова средняя и худшая сложность операций в хеш-таблице?
11. Когда эффективнее использовать двоичный поиск, а когда - хеш-таблицу?
12. Что такое коэффициент загрузки хеш-таблицы? Почему он важен?
13. Как работает рехеширование (динамическое расширение хеш-таблицы)?
14. Как выбрать размер хеш-таблицы? Почему часто выбирают простое число?
15. Как реализовать хеш-функцию для строк?

Требования к отчёту

Титульный лист
Цель работы
Теоретическая часть (двоичный поиск, хеш-таблицы, методы разрешения коллизий)
Постановка задачи (согласно варианту)
Листинг программы с комментариями
Результаты работы (скриншоты вывода)
Сравнительная таблица производительности
Ответы на контрольные вопросы
Выводы
Ссылка на Git-репозиторий

Критерии оценивания
Максимальный балл: 100

№	Критерий	Баллы
1	Двоичный поиск	25
1.1	Реализация алгоритма	10
1.2	Корректность на граничных случаях	10
1.3	Обработка ошибок (элемент не найден)	5
2	Хеш-таблица	40
2.1	Реализация хеш-функции	10
2.2	Реализация метода разрешения коллизий	15
2.3	Реализация insert, find, remove	15
3	Сравнительный анализ	20
3.1	Измерение времени выполнения	10
3.2	Таблица/график сравнения	5
3.3	Выводы о производительности	5
4	Оформление и отчёт	15
4.1	Код отформатирован, есть комментарии	5
4.2	Есть ответы на контрольные вопросы	5
4.3	Ссылка на Git-репозиторий	5
Итого		100

Штрафные баллы

Нарушение	Штраф
Неверная сложность алгоритма	-10
Плохая хеш-функция (много коллизий)	-10
Утечка памяти (в методе цепочек)	-10
Нет сравнительного анализа	-10
Нет ссылки на Git-репозиторий	-10
Программа не компилируется	работа не принимается

Итоговая шкала

Оценка	Баллы
Отлично	90–100
Хорошо	75–89
Удовлетворительно	60–74
Неудовлетворительно	менее 60